# SYSTEM ANALYSIS DOCUMENTATION (SAD)

## Finite Automata (AI Fridge)

## UMHackathon 2026

umhackathon@um.edu.my

---

© UMHackathon 2026

Email: umhackathon@um.edu.my

## 1. Introduction:

This section introduces the strategic solution to the problem of household food waste highlighted in the Product Requirement Document.

### 1.1. Purpose:

This system analysis document highlights the technical scope and design-related decisions behind the AI Fridge development.

So, the key element of system that has been covered are as followed:

1. **Architecture**:
    a. A cloud-native platform deploying separated microservices such as OCR, inventory, recipe, and notification that work independently to handle agentic workflows.
2. **Data Flows:**
    a. How unstructured data like receipt images and voice notes flows from the user's mobile device, moves through the OCR/NLP backend API, gets stored relationally in PostgreSQL, and outputs structured JSON inventory lists.
3. **Model Process:**
    a. Shows the end-to-end workflow of core features: scanning a receipt, extracting items, managing expiration states, and suggesting recipes.
4. **Role of Reference:**
    a. This document serves as a master reference for our 5-member development and QA team to enable the overall high-level architecture of this project.

### 1.2. Background:

Households waste 30-40% of purchased food due to poor inventory tracking. Existing solutions are heavily manual, leading to rapid user drop-off.

1. **Previous Version:**
    a. Existing tools for grocery data, kitchen inventory, and meal planning operate in isolated silos. Users must juggle multiple platforms, requiring immense manual discipline.
    b. Existing tools don't really integrate the use of Artificial Intelligence for recipe suggestions efficiently. Users are just given certain options for communication with the recipe suggestor.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. **Changes in Major Architectural Components:**
   Transitioning from traditional, static CRUD (Create, Read, Update, Delete) databases to an event-driven, multi-agent microservice architecture. This shifts the system from relying on manual user inputs to utilizing an LLM which is Z.AI GLM as a core service layer for automated reasoning and data extraction.

3. **New capabilities that are only introduced in this project:** Integrating OCR (Optical Character Recognition) and NLP (Natural Language Processing) to completely eliminate manual data entry, alongside a proactive state machine that acts on expiring items and dynamically generates personalized recipes with and without user prompting.

## 1.3. Target Stakeholder

| Stakeholders | Roles | Expectations |
|---|---|---|
| Household Users | Scans receipts, manages physical inventory, cooks suggested meals using the mobile platform. | Frictionless automated tracking, reliable expiration alerts, and practical recipes using available ingredients. |
| Development Team | Builds the multi-agent platform and maintains the system architecture. | Clear API usage contracts, modular codebase where it is separated between mobile and web logic, and well-documented relational schemas. |
| QA Team | Validates system behavior across input types such as blurry receipts, text fallbacks etc. | Defined testing scope for the state machine, API request/response JSON formats, and coverage of OCR edge cases. |

© UMHackathon 2026

# 2. System Architecture & Design

## 2.1. High Level Architecture

### 2.1.1. Overview:

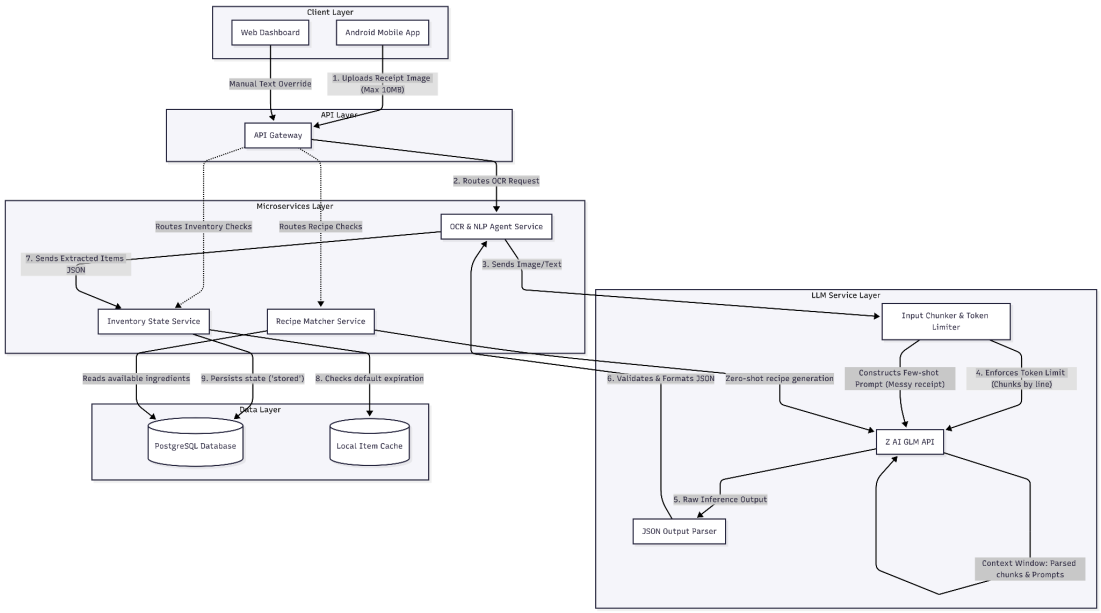
| Type | Details |
|---|---|
| System | Mobile App (Android Native) & Web Dashboard |
| Architecture | Cloud-Server with AI-driven Microservices |

AI Fridge is structured as a client-server-based system. The Mobile Application communicates via a Backend API layer to independent microservices: OCR Agent Service, Inventory State Service, and Recipe Agent Service. All microservices share a highly normalized relational database to maintain data integrity.

### 2.1.2 LLM as Service Layer

The architecture integrates the Z AI GLM model as a core service layer handling multi-agent workflows, rather than a generic "AI Module". The following Dependency diagram includes:

### 2.1.3 Dependency Diagram

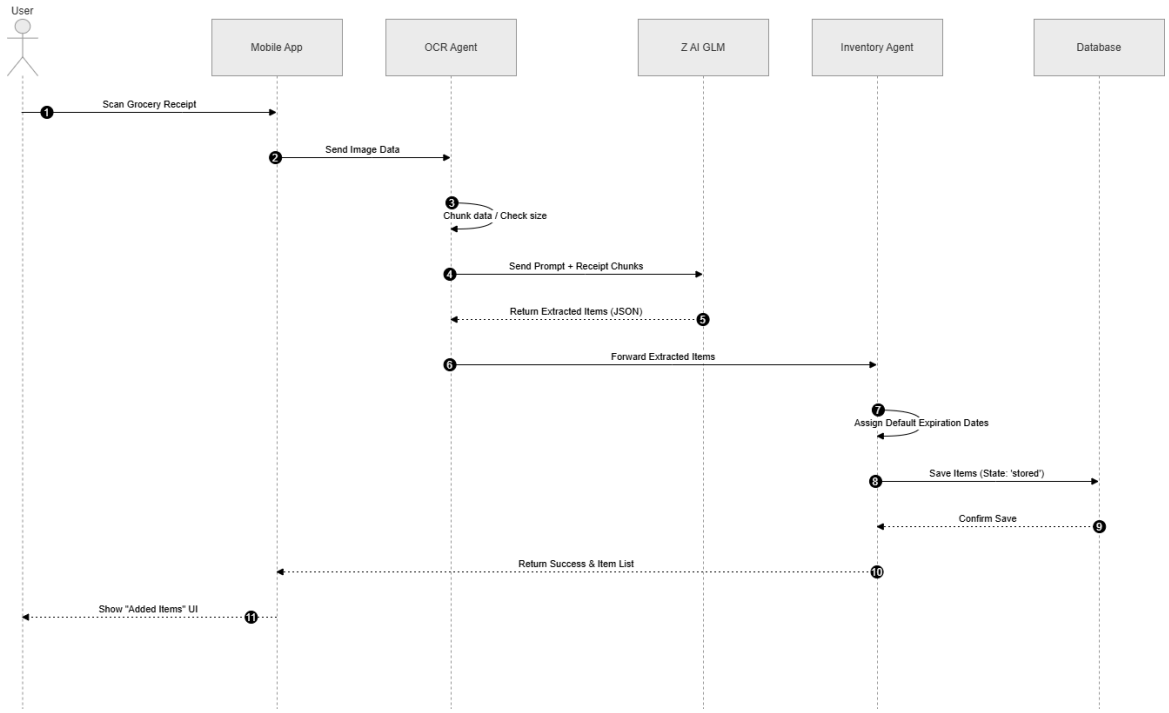



© UMHackathon 2026

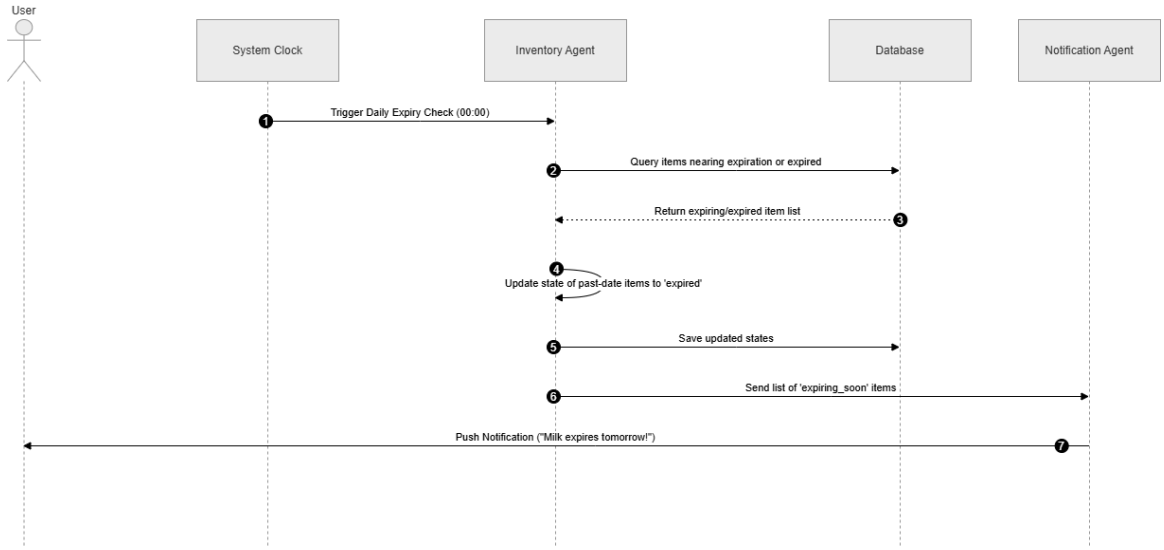
Email: umhackathon@um.edu.my

### 2.2.2. Sequence Diagram

1. Scan receipt and Extract item



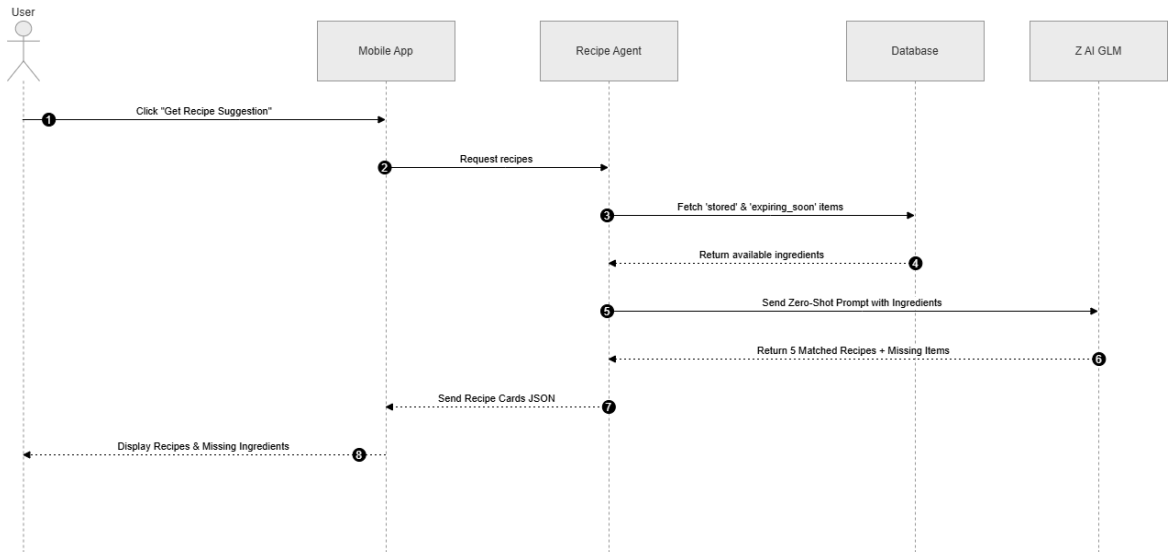

2. Expiration Alerts



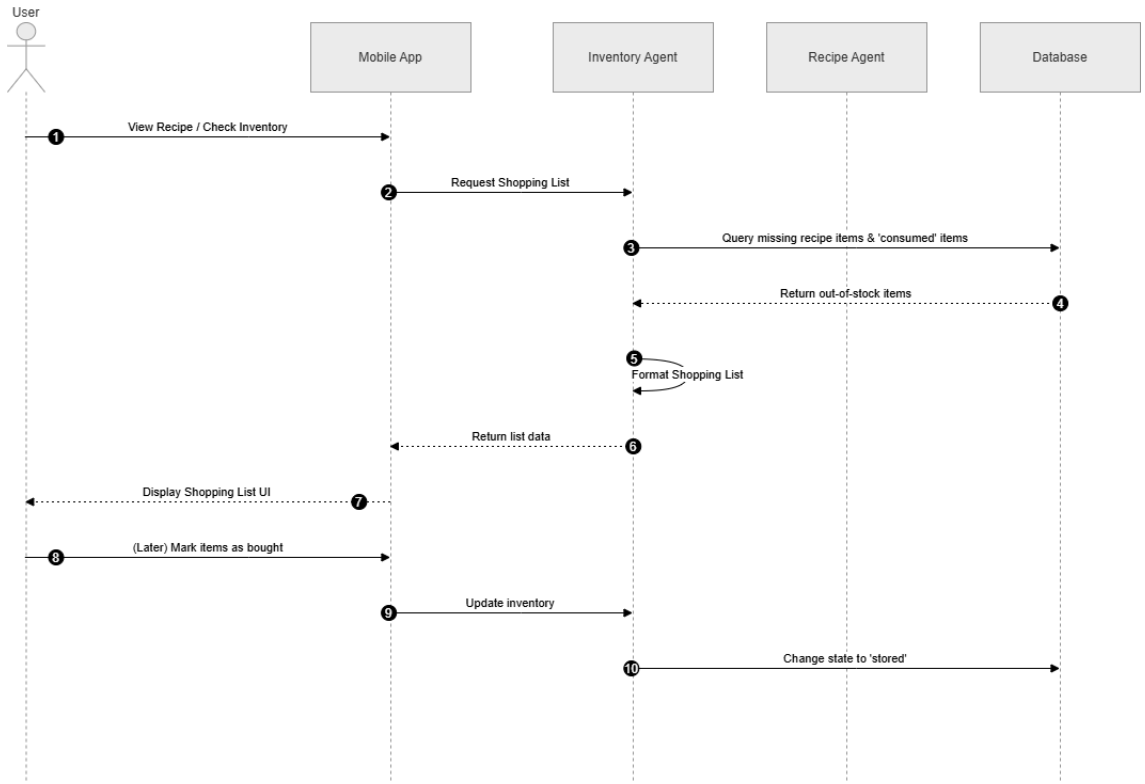

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Generate Recipe Suggestion




4. Generate Shopping List




© UMHackathon 2026

Email: umhackathon@um.edu.my

## 2.2. Technological Stack

**2.2.1.** Frontend: Java (Android Native) for mobile capture, JavaScript/HTML/Bootstrap for dashboard views.

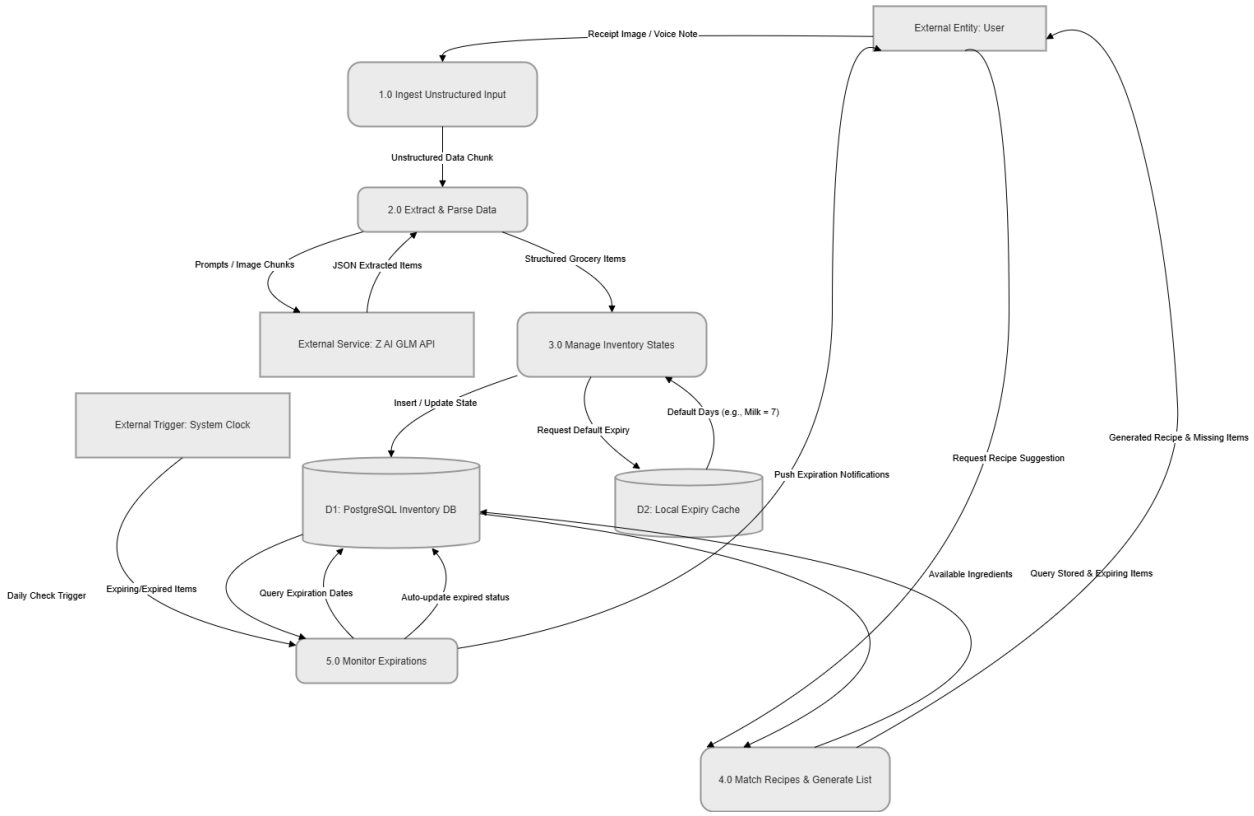
**2.2.2.** Backend: Node.js / Express for orchestrating RESTful microservices

**2.2.3.** Database: PostgreSQL – Relational Database designed with robust SQL joins for complex queries such as matching users, ingredient lists, and recipes.

**2.2.4.** Cloud/Deployment: Hosted server for API routing, static asset storage for recipe images.

## 2.3. Key Data Flows

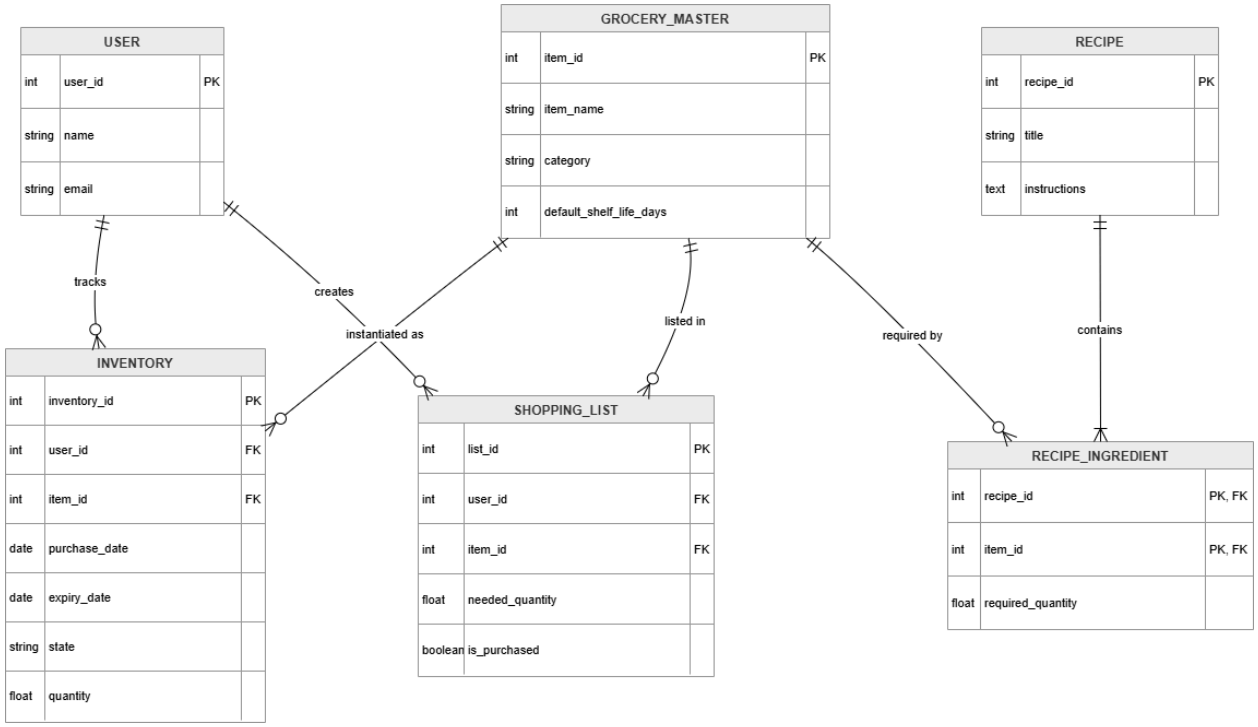
### 2.3.1.1. Data Flow Diagram (DFD)




© UMHackathon 2026

Email: umhackathon@um.edu.my

### 2.3.2. Normalized Database Schema




## 3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

## 3.1. Minimum Viable Product:

| # | Features | Description |
|---|---|---|
| **1** | Receipt OCR Extraction | Users upload receipt images. The OCR Agent automatically extracts items and quantities, handling blurry or messy inputs. |
| **2** | Inventory State Machine | Items are automatically tracked through specific lifecycle states:<br>1. Purchased<br>2. Stored<br>3. Expiring_soon<br>4. Expired<br>5. Consumed |

© UMHackathon 2026

| | | |
|---|---|---|
| **3** | Recipe Matching Engine | The system matches store and expiring_soon items to a whitelist of recipes, generating shopping lists for any missing ingredients. |

## 3.2. Non-Functional Requirements (NFRs)

| **Quality** | **Requirements** | **Implementation** |
|---|---|---|
| Scalability | The system must handle daily cron-job spikes when checking expiration states across all users. | Utilize background task queues such as BullMQ to distribute the processing load. |
| Reliability | Fallback mechanisms must prevent system crashes if OCR returns hallucinated items. | Graceful error prompts allowing users to manually type items via a fallback UI. |
| Token Latency | API calls to Z AI GLM must not bottleneck the user experience. | Implement a 5-second timeout for async calls, displaying an active loading state while parsing. |
| Cost Efficiency | Token costs must stay under the approximation of 5,000 to 8,000 tokens per session budget. | Cache frequently purchased items locally so default expiration dates bypass the LLM entirely. |

## 3.3. Out of Scope / Future Enhancements

a. Multi-user household sharing and synchronization.
b. Integration with physical smart fridges or IoT sensors.
c. Real-time automated grocery ordering and delivery integration.

© UMHackathon 2026

# 4. Monitor, Evaluation, Assumptions & Dependencies

## 4.1. Technical Evaluation:

**4.1.1. Grayscale Rollout & A/B Testing**: We will test recipe generation constraints initially on a small subset of test accounts, tweaking the zero-shot prompts before full integration.

**4.1.2. Emergency Rollback (ER):** If the OCR Agent pipeline begins failing due to an external API update, we will immediately revert to the manual-entry-only UI (Golden Release) to maintain app stability.

### 4.1.3. Priority Matrix:

**4.1.3.1. P1 Critical:** If Z AI GLM parsing failures exceed 5%, trigger emergency fallback to manual entry modes.

## 4.2. Monitoring:

**4.2.1.** The system will log instances where users have to manually override the "ambiguous item" prompt like generic "bakery item". High rates of manual override will trigger a review of our chain-of-thought prompting.

**4.3. Assumptions:** State key operational & environmental conditions that you assumed to be valid while development of system and deployment. [e.g.

- **a.** Users have decent lighting and a camera capable of capturing somewhat legible receipt images.
- **b.** Users will engage with the initial confirmation screen to correct any minor OCR hallucinations.

© UMHackathon 2026

### 4.4. External Dependencies:

| Tools | Purpose | Risks |
|---|---|---|
| Z AI GLM API | Core reasoning engine for OCR, NLP multi-step extraction, and structured output generation. | High risk if API limits are hit. Mitigation: Heavy reliance on cached items and a manual input fallback UI. |

## 5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

### 5.1. Project Timeline:

**Day 1-2**: Architecture and ERD Schema Planning;
**Day 3-7**: Agent Pipeline and Full-Stack Development;
**Day 8-10**: Quality Assurance, API optimization, and Demo Preparation.

### 5.2. Team Members Role:

**Fathi – Project Manager & QA Lead:** Coordinates task distribution using agile tracking, manages risk mitigation, and ensures the system architecture complies with established software process and quality standards like ISO 25010.

**Afiq – Mobile Frontend Developer:** Leads the native Android application development using Java, focusing heavily on building a frictionless camera interface for receipt scanning and seamless user flows.

**Fahim – Backend & Database Lead:** Handles the backend API routing and is strictly responsible for the PostgreSQL relational schema design like maintaining 3NF and writing SQL joins required to match inventory items to recipe requirements.

**Syameer – AI & Multi-Agent Engineer:** Focuses entirely on the Z AI GLM integration, managing the prompt engineering for the OCR and NLP agent pipelines, and building the fallback UI logic for hallucinated items.

**Faris – Web Dashboard UI Developer:** Develops the responsive web-based dashboard using HTML, JavaScript by handling DOM manipulation for dynamic inventory updates, and Bootstrap, while also utilizing AI tools to generate the app's visual design assets.

**5.3. Recommendations:** To ensure system cost-efficiency and speed, we recommend caching highly repeatable, low-complexity queries like standard expiration dates for "eggs" or "milk" locally, minimizing external API calls.

© UMHackathon 2026

© UMHackathon 2026

Email: umhackathon@um.edu.my